

Lab 5: Memory Request Scheduling

Introduction

Task 1: Getting Your Hands on Ramulator 2

Question 1: Baseline System Configuration

DRAM Architecture & Organization

- Standard: DDR4
- Organization Preset: DDR4_8Gb_x8 (8 Gigabit density per chip, with an 8-bit data interface).
- Channels: 1 channel.
- Ranks: 1 rank per channel.

Timing & Performance

- Timing Preset: DDR4_2400R (Operating at 2400 MT/s).
- Clock Ratio: 3 (The memory system operates at a frequency relative to the simulation base clock). Memory Controller Logic
- Controller Implementation: Generic
- Scheduling Policy: FRFCFS (First-Ready, First-Come-First-Serve), which prioritizes requests that hit an open row (row-buffer hits) and then older requests.
- Refresh Management: AllBank (Refreshes all banks simultaneously).

System Integration

- Address Mapping: RoBaRaCoCh (Row-Bank-Rank-Column-Channel). This mapping scheme determines how physical addresses are translated into DRAM coordinates, prioritizing row locality.
- Memory System Implementation: GenericDRAM.
- Frontend Connection: It is connected to a SimpleO3 (Out-of-Order) CPU model with a 64KB Last Level Cache (LLC).

Question 2

- The frontend's YAML configuration accepts a list of traces. Each trace in the list corresponds to a CPU core. The SimpleO3 frontend simulation length is determined by the number of retired instructions, not a fixed number of CPU cycles.
- The simulation terminates once all cores have retired the specified number of instructions.

Question 3

I ran one multicore simulation (2C L-Trace, 2C H-Trace) and two single core simulations (once 1C L-Trace, once 1C H-Trace) for 300K instructions per simulation. The number of CPU cycles per core can be gathered from the `cycles_recorded_core_x` statistic printed. The IPC is derived as:

$$\text{IPC} = \text{num_expected_insts} / \text{cycles_recorded_core_X}$$

Trace Type	Single-core IPC	Multi-core IPC	Diff
High Intensity (H)	0.602	0.416	-31%
Low Intensity (L)	3.22	2.174	-32%

Question 4

The TraceRecorder plugin is a memory controller plugin designed to log a cycle-accurate trace of all DRAM commands issued by the controller to a file. It creates a per-channel trace file at the path specified in the configuration. Every time the memory controller issues a command, the plugin records a new entry containing the following data:

- Timestamp
- Command Name
- Address Vector

Question 5

1. The Difference Between FCFS and FRFCFS

- **FCFS (First-Come-First-Serve):** Prioritizes requests solely based on when they arrived at the memory controller. If the oldest request is a row-buffer miss, the controller waits for the necessary PRE and ACT commands to complete before issuing any other requests, even if newer requests in the buffer are row-buffer hits.
- **FRFCFS (First-Ready, First-Come-First-Serve):** Adds a “First-Ready” prioritization layer. It first checks if any requests are “ready” (i.e., they are row-buffer hits and satisfy all timing constraints). It prioritizes these ready requests over older requests that are not yet ready. It only falls back to arrival time (FCFS) if multiple requests are ready or if no requests are ready.

2. Impact on System Performance The impact is significant, as seen in the multicore simulation results in the same setup as question 3:

Metric	FRFCFS Policy	FCFS Policy	Performance Impact
Memory System Cycles	270,168	789,751	~2.9x Slower
Core 0 Cycles (L-trace)	134,055	344,892	~2.6x Slower
Core 2 Cycles (H-trace)	720,447	2,097,535	~2.9x Slower

Citations